Ahsanullah University of Science and Technology
Department of Computer Science and Engineering

Experiment No 2

"Implementing the Perceptron algorithm for finding the weights of a Linear Discriminant function."

Pattern Recognition Lab
CSE – 4214

Submitted By
Rashtab Mahmud Shuvo
ID: 12.02.04.040
Year: 4th
Semester: 2nd
Section: A
Group: A2

Implementing the Perceptron algorithm for finding the weights of a Linear Discriminant function

Rashtab Mahmud Shuvo
Department of Computer Science and Engineering
Ahsanullah University of Science and Technology
Dhaka, Bangladesh

Objective

The main objective of this assignment is to implement the Perceptron algorithm for finding the weights of a Linear Discriminant function.

1. Introduction

Perceptron algorithm:

In machine learning, the **perceptron** is an **algorithm** for supervised learning of binary classifiers: functions that can decide whether an input (represented by a vector of numbers) belongs to one class or another.

The perceptron algorithm was invented in 1957 at the Cornell Aeronautical Laboratory by Frank Rosenblatt, funded by the United States Office of Naval Research.

2. Tasks

Given the following sample points in a 2-class problem:
w**1** = (1, 1), (1, -1), (4, 5)
w**2** = (2, 2.5), (0, 2), (2, 3)

1. Plot all sample points from both classes, but samples from the same class should have the same color and marker. Observer if these two classes can be separated with a linear boundary.
2. Consider the case of a second order polynomial discriminant function. Generate the high dimensional sample points y, as discussed in the class. We used the following formula:

   $$y = [\ x_1^2 \quad x_2^2 \quad x_1 * x_1 \quad x_1 \quad x_2 \quad 1]$$

   Also, normalize any one of the two classes.
3. Use Perceptron Algorithm (one at a time) for finding the weight-coefficients of the discriminant function (i.e., values of w) boundary for your linear classifier in question Here α is the learning rate and $0 < \alpha \leq 1$.

3. Implementation

Task 1

```
%---Task 1- plotting samples----%

class1 = [1 1; 1 -1; 4 5];
class2 = [2 2.5; 0 2 ; 2 3];

plot(class1(:,1), class1(:,2),
'bs');
grid on;
hold on;

plot(class2(:,1), class2(:,2),
'r*');
hold off;
```

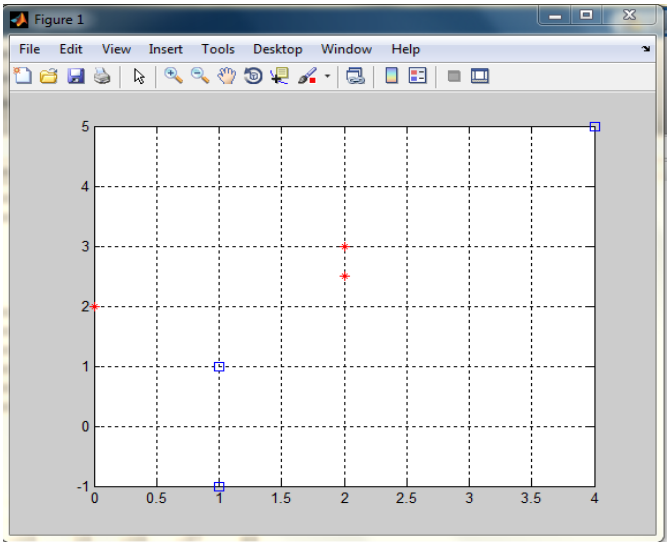

Figure 1: Output of Task 1

Task 2

```
%------Task-2------%

%--generating high dimensional y-%

y11 = [ class1(1,1)*class1(1,1)
class1(1,2)*class1(1,2)
class1(1,1)*class1(1,2) class1(1,1)
class1(1,2) 1];


y12 = [ class1(2,1)*class1(2,1)
class1(2,2)*class1(2,2)
class1(2,1)*class1(2,2) class1(2,1)
class1(2,2) 1];


y13 = [ class1(3,1)*class1(3,1)
class1(3,2)*class1(3,2)
class1(3,1)*class1(3,2) class1(3,1)
class1(3,2) 1];


%------normalizing class 2------%

y21 = [ -class2(1,1)*class2(1,1) -
class2(1,2)*class2(1,2) -
class2(1,1)*class2(1,2) -
class2(1,1) -class2(1,2) -1];

y22 = [ -class2(2,1)*class2(2,1) -
class2(2,2)*class2(2,2) -
class2(2,1)*class2(2,2) -
class2(2,1) -class2(2,2) -1];

y23 = [ -class2(3,1)*class2(3,1) -
class2(3,2)*class2(3,2) -
class2(3,1)*class2(3,2) -
class2(3,1) -class2(3,2) -1];


yAll=[y11; y12; y13; y21;y22;y23;];


display(y11);
display(y12);
display(y13);
display(y21);
display(y22);
display(y23);
```

Task 3 (one at a time)

```
%------Task-3-------------%
%------ONE AT A TIME------%
w = [1 1 1 1 1 1];
alpha = 1;
counter = 0;
flag = 0;
for j = 1:111111
    flag=0;
    counter = counter+1;
    for j = 1:6
    g = yAll(j,:)*w' ;
        if g < 0
            w = w + alpha *
yAll(j,:);
        else
            flag = flag + 1;
        end;
    end;

    if (flag == 6)
        break ;
    end;
end
fprintf('During One at time,
Counter is at %d\n', counter);
```

```
%---MANY AT A TIME----%
w = [1 1 1 1 1 1];
alpha= 1;
flag = 0;
counter = 0;
temp = 0;

for i = 1:111111
    counter = counter + 1;
    flag = 0;
    for j = 1:6
        g = yAll(j,:)*w' ;
        if g < 0
            temp = temp + yAll(j,
:);
        else
            flag = flag + 1;
        end;
    end;
    if (flag == 6)
        break ;
    end;
    w = w + alpha * temp;
end;

fprintf('During many at a time,
counter is at %d\n', counter);
```

Implementing the Perceptron algorithm for finding the weights of a Linear Discriminant function

Task 4

```
%--------Task-4------%
hold on;

syms class1 class2;

s=sym(w(1,1)*class1*class1+w(1,2)*
class2*class2+w(1,3)*class1*class2
+w(1,4)*class1+w(1,5)*class2+w(1,6
));

s2=solve(s,class2);

xvals1=[-10:0.01:10];
xvals2(1,:)=subs(s2(1),class1,xval
s1);

plot(xvals1,xvals2(1,:),'k');
grid, hold,
```

So the final output of the program is given in the figure below:

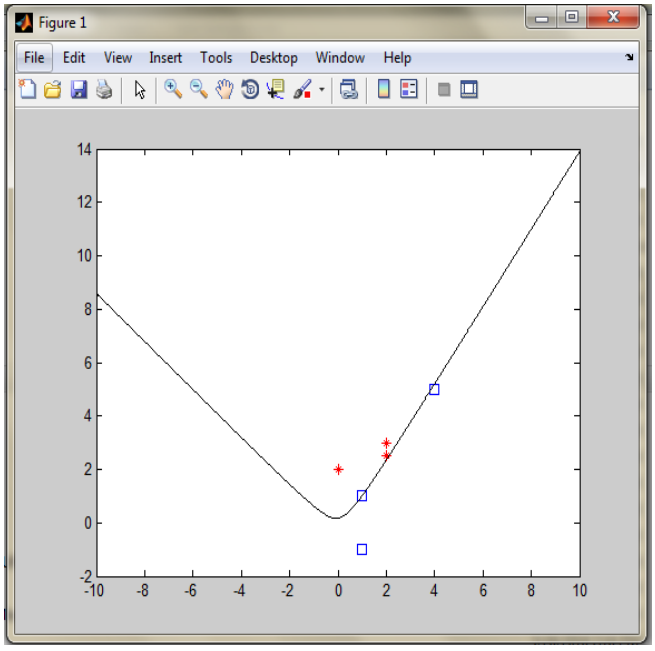

Figure 2: Final Output of the experiment with decision boundary

4. Case Study

In this experiment we have dealt with two given class. In my case study, I have changed the value with 2 new classes where

```
class1 = [1 2; 0 -1; 3 3];
class2 = [0 2; 0 5; 2.5 3];
```

Using these two classes and using the same code we can find the final output like so:

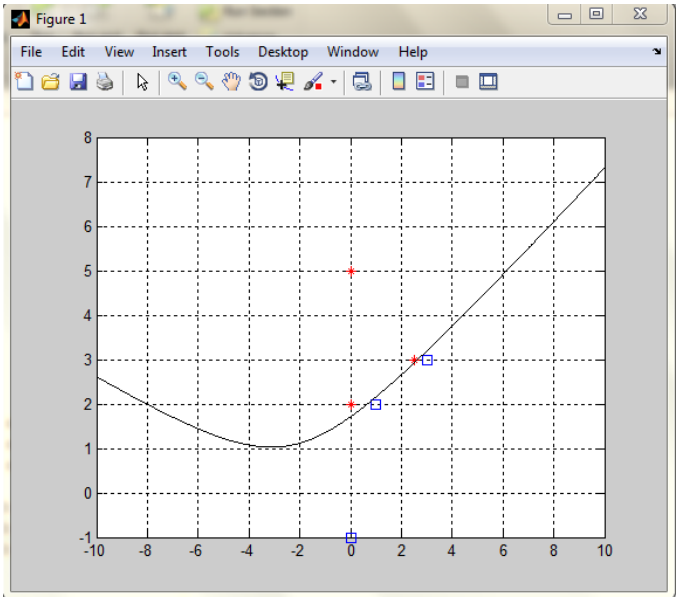

Figure 3: Case Study with new values

In this case, the final result is quite different than the previous one as I have explained in the result analysis below.

5. Result Analysis

In case of Task 1, I have followed standard procedure to plot the given points into a grid system as I have shown in figure 1.

In case of Task 2, for the second order polynomial discriminant function, we have used the following formula

$$y = [\, x_1^2 \;\; x_2^2 \;\; x_1 * x_1 \;\; x_1 \;\; x_2 \;\; 1].$$

In case of task 3, there are two methods where with which we can finish the experiment, One at a time approach & many at a time approach. . The efficiency can be measured if we follow the number of loops needed to complete the task.

For the given experiment we have found that the loop number is **94** for One at a time approach & **67** for many at a time approach.

For my own case study, I have found that the loop number is **68** for One at a time approach & **46** for many at a time approach.

6. Conclusion

In this experiment we conclude the Perceptron Algorithm with two different approach taken where in both case studies show essence of the algorithm by drawing a decision boundary with given class problems with a learning curve of Alpha (=1) and we have also seen that in both cases, many at a time approach is better.